

SHUOHUI LIU

MSE800_F_Assessment1_OOP_Software_Engineering

 Assessment-1

Document Details

Submission ID

trn:oid::3618:151572860

Submission Date

Sep 13, 2026, 9:16 PM GMT+12

Download Date

Sep 13, 2026, 9:24 PM GMT+12

File Name

MSE800_F_Assessment1_OOP_Software_Engineering.pdf

File Size

950.3 KB

11 Pages

1,828 Words

12,166 Characters

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

-  **4 Not Cited or Quoted 5%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2%  Internet sources
- 1%  Publications
- 4%  Submitted works (Student Papers)

Match Groups

- 4 Not Cited or Quoted 5%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 2% Internet sources
- 1% Publications
- 4% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers		
		Yoobee on 2026-05-23	2%
2	Internet		
		www.bom.gov.au	1%
3	Internet		
		ksmith2.teammetro.net	<1%



MSE800 Professional Software Engineering

Assessment 1: Object-Oriented Programming Assessment

Student ID: 270926691

Student Name: Shuohui Liu

Date: 11 September 2026

1. Introduction	2
2. System Design.....	3
2.1 Layered Architecture	3
2.2 Application Layer	4
2.3 Business Service Layer & Micro-Service Layer	6
2.4 Storage Layer.....	8
3. Innovation Spec.....	9
3.1 One-step Picking-up-Returning System (OPRS).....	9
3.2 Working Process	9
3.3 Limitation	10
4. Efforts on Maintenance	10
5. Software Evolution.....	10
5.1 Evolution Over Time	10
5.2 Development Enhancement	10
5.3 Importance of Software Engineering Principles	11
6. Conclusion.....	11

1. Introduction

This report is an explanation of Automated Car Rental System (ACRS) that is a system enhanced traditional process of car rental business, storing and managing the information of customers, administrators, cars, booking actions and orders history. Moreover, the system contains one innovative spec that enables customers to automatically pick up the car they booked, no longer needing a person to assist with the car picking-up process, easily scanning a QR-code as instead, which lowers the workload of employers and promotes the working efficiency and economy profits.

This project applies multiple appropriate design patterns, such as Builder, Singleton, and Observer into the system design, which makes the structure of the project clear and easily understandable, and keeps the modularity and maintainability enhanced. And the innovative spec is designed by IoT technology with a one step process.

In this paper, some improvements are also provided in the last part, including a plan to outline the evolvement of the ACRS, ways to handle updates, bug fixes and new feature developments, and strategies for managing software maintenance, versioning, and backward compatibility.

Until now, this project mainly focuses on server side programmed in Python language for considering front-end as a temporary example for those who care about ACRS to have a visible user interface and experience about how the system works step by step. Therefore, this paper will introduce the back-end part majorly.

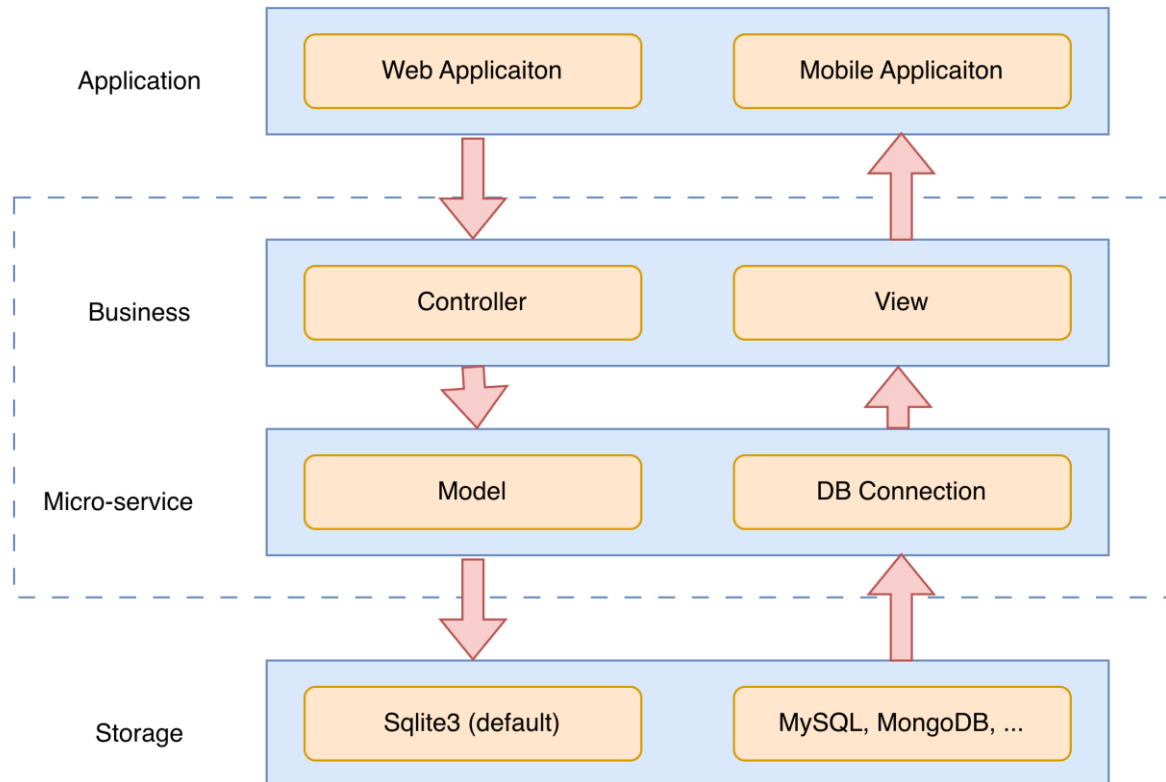
2. System Design

ACRS is designed by Client-Server architecture, having front-end to provide user with user interfaces and interactions and back-end to store information of Car, User and Order, and the system maintains a layered architecture to distinguish functions for each layer.

- User: The subsystem that stores and manages user information by two different roles: Admin and Customer.
- Car: A subsystem is used to store and manage all the cars for ACRS.
- Order: Also means the Booking subsystem, to store and manage booking history between users and cars.

2.1 Layered Architecture

ACRS is performed with 4 layers: Application Layer, Business Service Layer, Micro-service Layer and Storage Layer, structured by isolated working specifications, which shows as the bellow diagram:



The working process declares the two-way data transaction, which means while a user operates the website, data generated by the user is posted to invoke a specific business logic by invoking API, a controller receives the requests and divide it into several different working parts, each part invokes a particular model to operate database if needed, database returns the data back to model, and back to the controller, and the controller will composed all the data from models to a view and response it to front-end user. That is the whole working process of data operation.

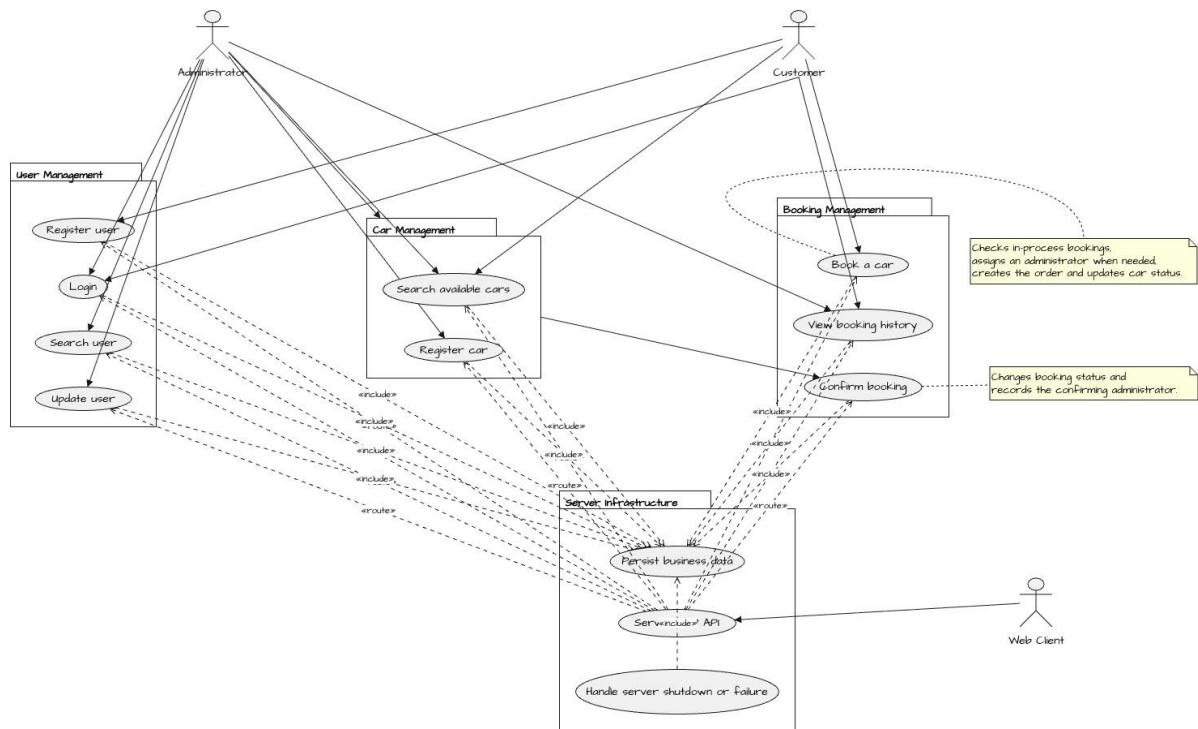
2.2 Application Layer

The application layer is the entrance for users to visit all the services provided by the ACRS with supporting multiple platforms, such as Web, H5, mobile platforms, etc. For current, it is only supportive to Web and H5, but mobile platforms and the others will be supported in the future because H5 can be directly applied to them by adding containers for each. Moreover, H5 is considered an efficient way to quickly adapt to any platform, to explore experimental and innovative applications, and to enhance the system's compatibility and flexibility.

Without considering the difference from crossing platforms, ACRS currently has supportive functions as bellow:

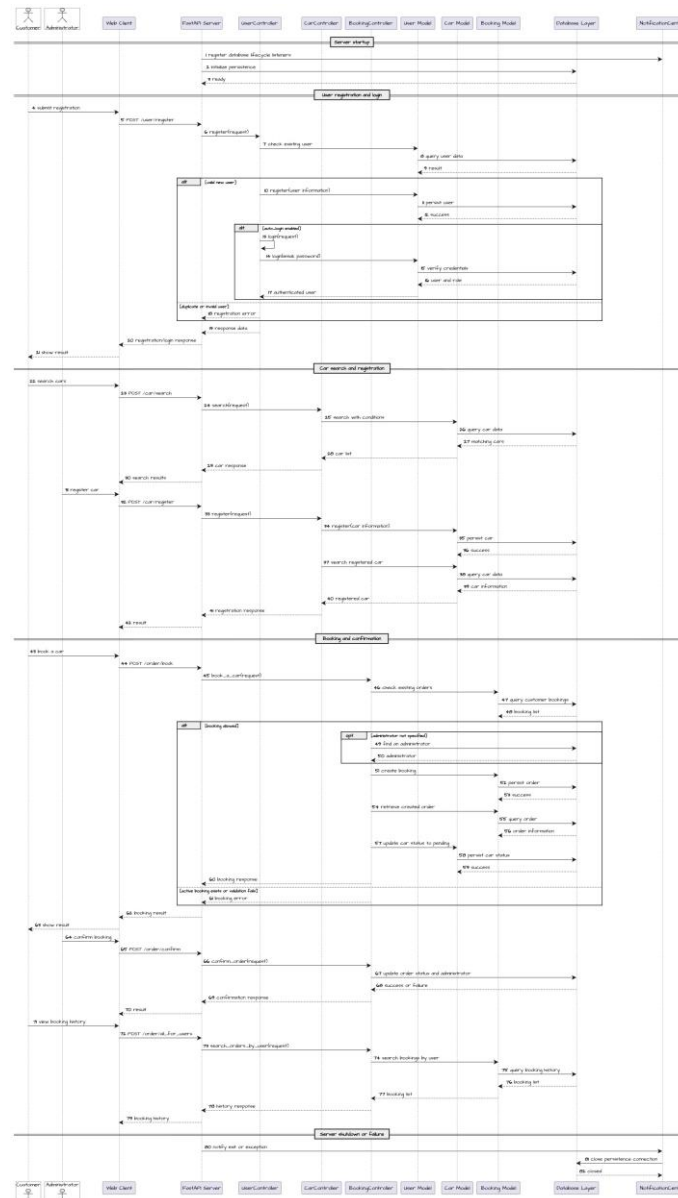
- User-Model: Login, register, and user role selection,
- Car-Model: Register cars by admins, show all the cars, update cars info (such as mileage, car status etc.), and
- Booking-Model (Order-Model): Book a car for a customer, operate the booking for an administrator, pick-up and return a car, etc.

Use Case Diagram:



Ref: [Serve - Use Case Diagram](#)

Sequence Diagram:



Ref: [Server - Sequence Diagram](#)

2.3 Business Service Layer & Micro-Service Layer

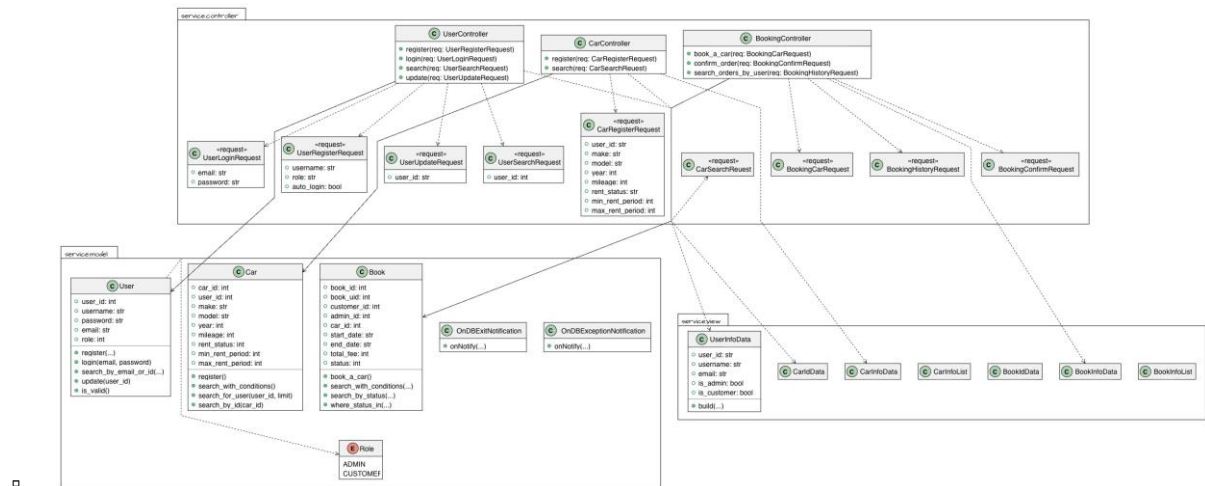
These two layers combine together to explain a whole MVC (Model-View-Controller) structure.

- Model: to keep connection to database, operate it and format the data to Controller,
- View: to compose and store the information needed to display on front-end, and
- Controller: To organize the business by composing multiple parts of data from the Model with logic and outcoming View data structure.

Business Service Layer contains both the Controller and View to handle user API requests and to respond to the composed info to Application Layer to apply, by gathering data from multiple

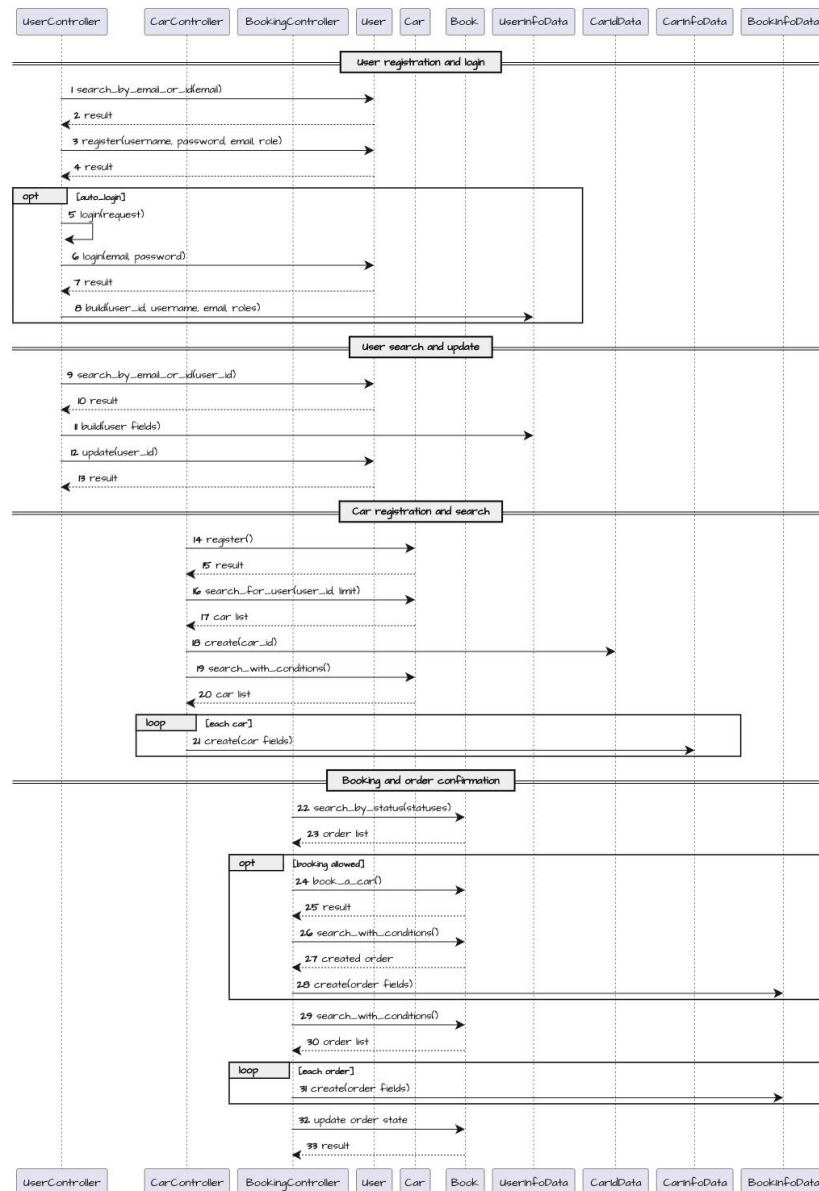
Model and organizing all data to be responsible View by business logic. The following show how MVC is designed with UML diagrams:

Class Diagram:



Ref: [Server/Service - Class Diagram](#)

Sequence Diagram:

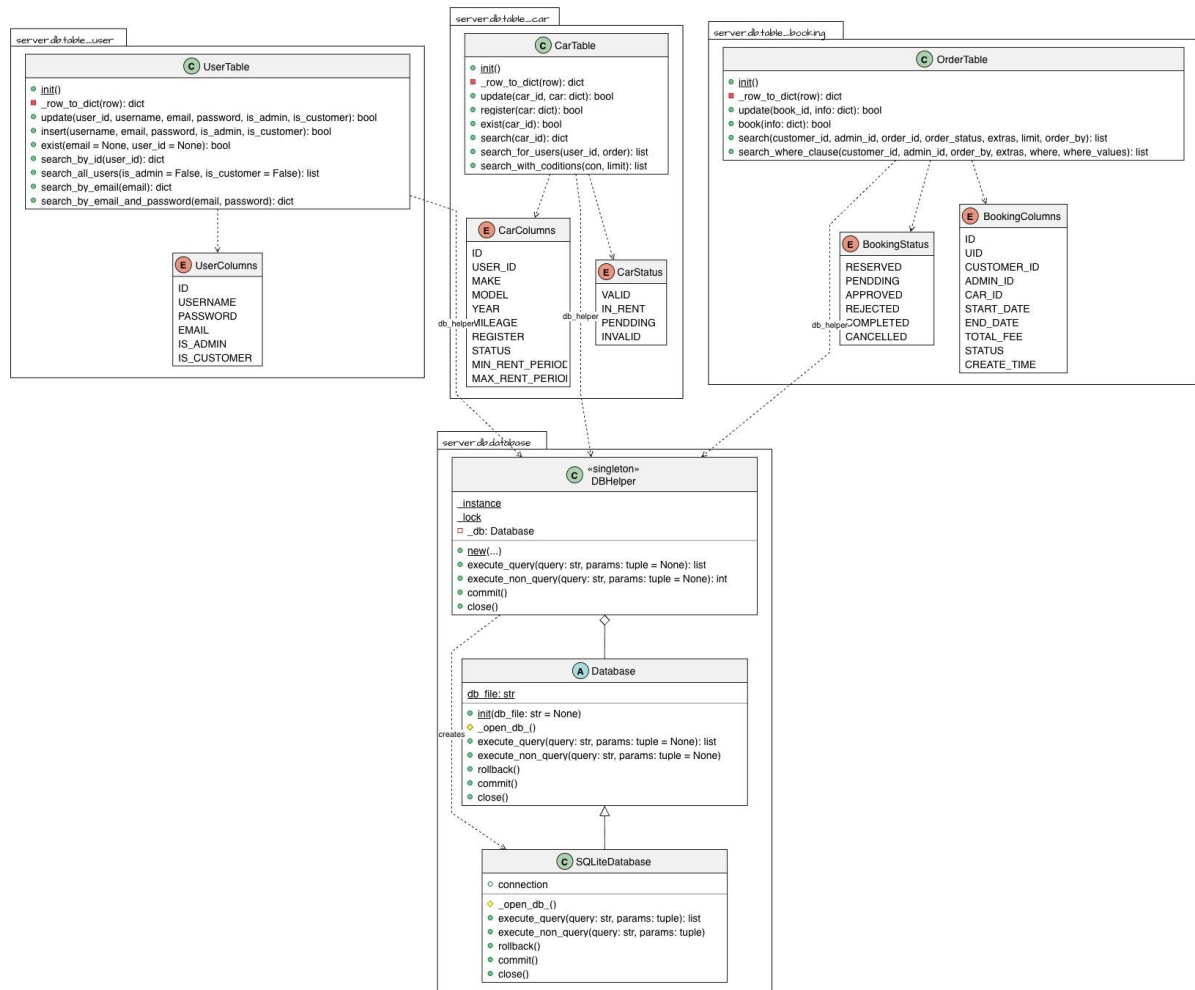


Ref: [Server/Service - Sequence Diagram](#)

2.4 Storage Layer

This layer contains atomic functions to provide database operations separated by 3 different isolated models: User, Booking, and Cars. There are no interactive operations between them, so only Class Diagram is needed to explain for this part to claim how many classes are defined.

- Class Diagram:



Ref: [Server/DB - Class Diagram](#)

3. Innovation Spec

3.1 One-step Picking-up-Returning System (OPRS)

OPRS is a function to provide customers with one-step way to pick up cars they booked and to return cars when they finish using the cars. This system is designed with IoT and QR-Code technology. QR-Code can be scanned by specific IoT devices or mobile applications, by which workload of employers is reduced.

3.2 Working Process

The designed working process is:

- Generate an API method to support this one-step method once a car is confirmed by an administrator to be ready to pick up,
- The customer refreshes the order which will have a QR-Code combined with the API method displayed in the order,
- The QR-Code is scanned by an administrator system (could be an IoT device or something else),
- Then the API method is invoked to complete the picking-up process.

- A returning API method is assigned to returning service, which can be invoked while returning the car,
- Another QR-Code is assigned to this order and display to the customer,
- Scan the QR-Code and complete the returning process.

3.3 Limitation

As the front-end application of this project plays the role of demo presentation, it limited the implementation of the whole OPRS. If it were applied to the production environment, the specific IoT devices or mobile applications should be developed to support it.

Another limitation is that this project is designed to support multiple databases; however, it currently supports Sqlite3 as one available execution.

4. Efforts on Maintenance

This project applies three design patterns to enhance maintenance and compatibility. They are:

- Builder: Be used to build responsible data elements to respond the API requests,
- Observer: Be used to register listeners and take specific actions for particular notifications. In this project, it applied to observer the execution of the server, and to take actions while receiving exceptions, such as close database no matter whether the server terminates normally or within exceptions, and
- Singleton: Be used to maintain a unique database connection.

Besides that, another effort is to support multiple databases to adapt different specifics of different databases. For this target, this project abstracts whole database operations and builds a complete logic to plug in a specific database scheme while running the service.

5. Software Evolution

5.1 Evolution Over Time

The ACRS should be evolved by researching and development the possibility and business application of most popular advanced technologies, such as IoT, AI etc. For example, in this project, IoT devices are designed to assist to promote the process of renting cars. As AI has strong capability of great model processing, it is valuable to research its application of suitable business scenes.

5.2 Development Enhancement

- Handle updates:
 - Database updates: using version control of database to alter fields of tables,
 - Backend updates: making version control for API methods and dispatching to different invokes,

- Front-end updates: for web applications, the updates are real-time when they are published; for mobile applications, that relate to the reinstallation of different versions, so backend version control of API methods will be quite significant.
- Bug fixes:
 - Backend: real-time fixes of bugs, the modification can be published directly without concerns about its reflections if it is an isolated modification. However, if it impacts other relatives, a logic of version control is necessary.
 - Frontend: web applications can be directly published when bugs are fixed; mobile applications can be designed to use H5 applications instead of issued function because it is unavailable to make real-time reinstallation.
- New feature additions: for backend, new features can be published any time with considering version control; for frontend applications, new features can be published as H5 supportiveness on mobile devices to collection running statistics; and once they are stable, transform H5 application to systematic performance if needed.
- Versioning:
 - Backend: using version filed for each API method or adding version control logic for programming; and adding statistics logic for each version of API invokes, which is used to delete a version logic or branch.
 - Config center: being used to dispatch different related participants to perform specific versions of API or something else according to its config details.
 - Frontend: designing version control architecture and implementation on specific systematic applications
- Backward compatibility: collecting statistics from all implementation participants and analyzing them to what functions can be removed. It is impossible to ensure a zero percent implementation of specific functions; there should be a verified standard value of the percent. And all participants directly interacting with users should have the logic of updating applications to notify users to make decisions.

5.3 Importance of Software Engineering Principles

Valuable software engineering principles can enhance the implementation of applications by making the system flexible and strong to perform. Different specifics of different systems and platforms lead the decision making of applying the particular principle from multiples to be serious and significant.

6. Conclusion

To conclude, this project completely achieves all the requirements of this assessment. And make efforts to enhance the maintenance and compatibility of ACRS. However, this project also has limitations of temporary implementation of front-end and QR-Code scanning as a working demo. Even though, this project still achieves its targets and has significant visions for the future Car Rental System.